VAAL UNIVERSITY OF TECHNOLOGY

Inspiring thought. Shaping talent.

FACULTY: Applied and Computer Sciences
DEPARTMENT: **Computer** Sciences
SUBJECT: Business Analysis 3.2
SUBJECT CODE: AIBUY3A
LECTURER:
YEAR: 2026

ASSESSMENT NAME:	Capstone Project
ASSESSMENT DATE:	02 November 2026

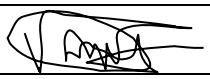
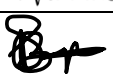
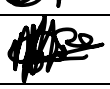
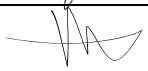
STUDENT NUMBER	SURNAME AND INITIALS	SIGNATURE
224844709	TLADI TDN	
224995782	Radebe H	
222382708	Dlamini V	
224784544	Ngcana S	
224649744	Linda W	
224703366	BALOYI SK	
223606766	Sewape J	
224829262	Mojalefa N	
225027380	Mhlaba V	

Table of Contents

Industry 2
AI Solution 2
Background 2
Business Objectives 3
Problem Definition 7
Poster 9
Machine Learning Approach 10
Data 11
Model 12
Time Series Analysis 16
Solution Techniques 17
Natural Language Processing 18
Deep Learning Techniques 19
Other Features 22
AI Solution Practical 23
DECLARATION STATEMENT 30
References 31

Industry

The logistics & Transportation industry often experiences delivery delays due to road accidents, traffic, severe weather conditions, and unexpected road closures. GPS navigation systems often recommend routes based on distance or travel time and may not consider all the factors that could affect the safety and efficiency of a journey, which may result in drivers encountering unforeseen obstacles that increase delivery times, fuel consumption, and operational costs.

AI Solution

Our company SafePath has developed an AI model for the Logistics & Transportation industry that scans different routes for accidents, traffic, and more. It simultaneously predicts and helps truck drivers avoid delays by redirecting them to the safest and most efficient route before problems arise. This will reduce travel delays, improve delivery reliability, decrease fuel usage, and support smoother transportation operations within local communities and municipalities.

Background

SafePath Logistics came about from the delivery challenges faced by carriers when delivering within our local communities, such as long delivery times and parcels and packages being stolen from delivery trucks. We built an AI solution/model that predicts routes that may cause delays and avoids them for better service delivery and the safety of drivers and goods delivered.

Business Objectives

Current navigation applications, such as Google Maps, optimize routing heavily around the shortest physical distance or historical travel time. However, they operate in separate functional silos and fail to deeply integrate critical safety parameters into their core calculations. Real-world disruptions like recent car accidents, sudden road closures, infrastructure damage (potholes), vehicle theft hazards, and severe weather conditions are treated as static alerts rather than path-blocking metrics.

For private motorists and commercial logistics companies, relying on distance-biased routing creates severe operational vulnerabilities. Drivers are frequently sent directly into bottlenecks or hazard zones. This results in extensive delivery delays, higher fuel costs due to idling in traffic, and an increased risk of vehicle damage or accident exposure. Implementing an AI-powered system that simultaneously scans and cross-references route conditions solves this gap by predicting the safest and most efficient path before problems arise.

Strategic Business Objectives

The core goals of this project are defined by six strategic pillars:

- **To Minimize Travel Time:** To reduce overall travel time for drivers by accurately predicting the shortest route based on real-time distance and traffic analysis.
- **To Enhance Driver and Passenger Safety:** To decrease the risk of accidents by avoiding routes with recent accidents, road closures, and hazardous weather conditions.
- **To Reduce Operational Costs:** To lower fuel consumption and vehicle maintenance costs for logistics and transport companies by optimizing routes.
- **To Improve Customer Satisfaction:** To provide a reliable and efficient navigation experience that builds user trust by prioritizing both speed and safety.
- **To Enable Data-Driven Decision Making:** To utilize continuous real-time data through a regression model to make dynamic and accurate route predictions.
- **To Increase Efficiency for Businesses:** To help delivery, e-hailing, and logistics companies improve on-time delivery performance and fleet management.

Requirements.

- **Car Accident Requirements:** The data pipeline must ingest real-time collision alerts and accident density scores per kilometer. The functional engine must immediately calculate delay coefficients around active accident zones, trigger alternative path evaluations, and display an immediate visual safety warning on the driver's interface.
- **Traffic Rate Requirements:** The system must stream live vehicle flow volumes (cars-per-minute) and average segment speeds. The ranking engine must use this continuous data stream to predict traffic bottlenecks, update the trip's estimated time of arrival (ETA), and initiate an automatic reroute pass every 30 seconds to bypass sudden gridlocks.
- **Pothole & Infrastructure Requirements**: The system must integrate municipal road quality reports and historical suspension-damage data into a continuous infrastructure severity index. The routing algorithm must heavily penalize and avoid pathways with high pothole density to lower fleet maintenance costs and protect vehicle assets.
- **Car Theft & Security Requirements:** The data architecture must sync with crime incident databases to pull historical and live vehicle theft indices. The system must automatically analyze route safety parameters based on the transit schedule, systematically filtering out high-theft sectors and high-crime routing vectors during vulnerable operating hours (such as night-time or long- haul deliveries).
- **Unified Processing Requirements:** The technical backend must utilize a cloud platform capable of running a supervised regression model. This model must process all four risk variables simultaneously, converting continuous data updates into a single "safest + shortest" route recommendation displayed on a low-distraction driver dashboard.

Constraints

- **Critical Dependency on External Data Feeds (APIs):** The system does not generate its own underlying mapping or risk data; it is entirely dependent on third-party APIs. It relies on platform networks like the Google Maps API for spatial distance matrices, HERE Technologies for real-time traffic volume data, and local municipal databases for infrastructure (potholes) and security (car theft) logs. Consequently, any backend service disruptions or downtime from these external providers will instantly degrade the system's ability to track active hazards.
- **Mandatory Network Connectivity and Live Telemetry:** To stream immediate accident alerts and fluid traffic rates, the application requires a continuous, high-speed mobile internet connection (4G/5G). The predictive regression pipeline operates on moving data; therefore, entering cellular dead zones or areas with poor network coverage will freeze real-time updates and force the system to rely on cached historical data.
- **High Computational Overhead and Resource Consumption:** Continuously calculating multi-variable regression scores across 3 to 5 alternative routes every 30 seconds demands significant processing power. This creates a high computational load on cloud servers and accelerates battery drain on mobile user interfaces and in-vehicle scanning machines if edge rendering is not heavily optimized.

RISKS

- **Safety Blind Spots from API Outages:** Because the system depends entirely on third-party APIs like Google Maps and HERE Traffic, any provider downtime blinds the system mid-trip. This prevents it from detecting live car accidents or closures, potentially routing drivers directly into unmapped danger zones.
- **Frozen Updates in Cellular Dead Zones:** The 30-second auto-reroute feature requires a continuous 4G/5G connection to stream live data. Dropping mobile signal completely freezes incoming data, making it impossible to track moving traffic rates or newly reported hazards until connectivity is restored.
- **Server Slowdowns During Peak Hours:** Scanning 3 to 5 alternative paths every 30 seconds for thousands of vehicles simultaneously creates massive

computational strain on cloud servers. During peak hours, this high data volume can cause processing delays, holding up critical route changes and warning alerts.

- **Secondary Jams on Minor Roads:** If the regression model identifies a specific local road as the single “safest” alternative to bypass an accident or an active car theft hotspot, it will aggressively route all fleet units onto that path, creating a massive, secondary traffic jam.
- **Privacy Vulnerabilities and Unbalanced Hazard Data:** Continuously tracking precise user coordinates to monitor local car theft risks exposes the platform to severe data privacy compliance threats. Additionally, if historical data for potholes and crime trends is unbalanced, the AI model will over- penalize safe communities and miscalculate baseline risks.

TOOLS

Implementation Tools

- **Python & Scikit-learn / XGBoost:** Used to build, train, and validate the regression models [Scikit-Learn Machine Learning in Python, XGBoost Documentation].
- **Google Maps / Mapbox API:** Integrated to pull alternative paths, road geometries, and baseline distance matrices.
- **TomTom / HERE API:** Used to capture live traffic speeds and real-time accident telemetry streams.
- **OpenWeatherMap API:** Used for live weather streaming, converting weather data into a continuous safety index.
- **Pandas & NumPy:** Deployed for real-time data cleaning, matrix calculations, and feature engineering.
- **Firebase / AWS:** Selected for cloud backend architecture, real-time data hosting, and low-latency deployment scaling.

Problem Definition

Companies involved in logistics and delivery drivers experience delivery delays, damage/road accidents, and increased operational costs due to road conditions. Existing navigation systems installed in today's world primarily consider distance and estimated travel time without sufficiently considering multiple factors that can affect the safety of a route. These factors include traffic volume, recent road accidents, severe weather, road restrictions, potholes, and other road-changing conditions. This results in delivery drivers being guided onto routes that are short in distance but jammed with traffic, routes that are also exposed to extreme weather conditions or restricted roads, causing longer delivery times, increased fuel consumption, and unreliable deliveries.

In the transportation and logistics industry, reliable road conditions are vital to ensure the safe arrival of goods to its destination in the expected time of delivery. However, the unpredictable road conditions that can change during the course of the trip make it a challenge for drivers to know the most suitable routes to take.

Recent road accidents can cause traffic buildup and delays in certain areas and temporary restrictions on certain roads. When there are severe weather conditions, like raining heavily and visibility is bad, this can make driving more difficult and dangerous. Potholes or damaged road can make vehicle at risk for being damaged or accident. So you'll end up with extra costs for maintenance and repair. Road closures or restricted access can make your destination not accessible by your intended route. These all can cause delivery drivers to take routes that are shorter but have traffic congestion, are affected by recent accidents, dangerous weather, or restricted access. This can make their journey take longer than expected. It can increase fuel consumption and un-reliable for delivery schedule.

Road conditions can impact the cost of logistics operations and bring about several challenges for logistics companies. The longer time spent on the road due to poor road conditions can lead to increased fuel consumption, higher vehicle operating costs, and decreased productivity as the number of deliveries made in a day might be reduced. Delays in delivering goods can also affect customer satisfaction and the reliability of the transportation services. If it happens that your vehicle gets damaged due to

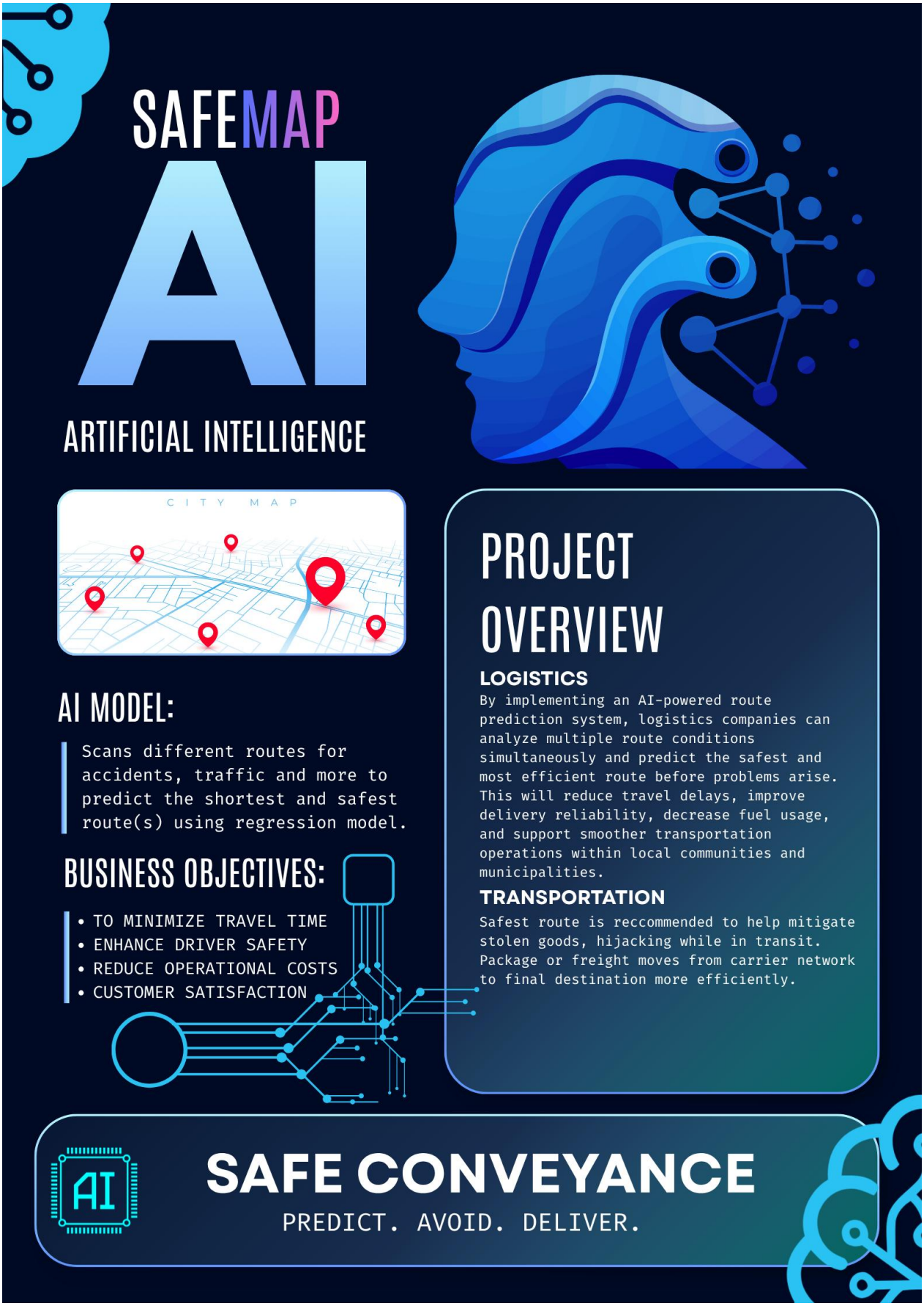
accidents or bad road conditions, there can be additional costs for repairs and disruptions in business operations.

Challenges like these highlight the impact of road conditions on the overall efficiency of logistics and the increased cost associated with the transportation of goods and services.

This challenge is relevant for AI solutions in the industries theme because it highlights an operational issue within the logistics and transportation sectors where presently installed navigational solutions do not adequately take into account all of the factors that impact the safety and efficiency of routing solutions. This issue industries face is also relevant to local municipalities, as inefficient routing solutions for logistics vehicles can contribute to traffic congestion issues, increase road safety risks, and add stress to municipal road infrastructure.

Unforeseen road accidents can affect private drivers, public transportation, and other road users. Delays affect the movement of goods and access to important destinations. When vehicles are caught in traffic, fuel is wasted unnecessarily and causes unwanted emissions that negatively affect environmental quality and health. Traffic congestion and road accidents can cause delays for emergency response vehicles, potentially impacting emergency service response time to those in need. It's important to address these challenges not only to improve logistics but also for safer roads, reduce unnecessary congestion, improve emergency response times, limit unwanted emissions from vehicles, and better environmental quality for citizens.

Poster


SAFEMAP
AI
ARTIFICIAL INTELLIGENCE
CITY MAP
AI MODEL:
Scans different routes for accidents, traffic and more to predict the shortest and safest route(s) using regression model.
BUSINESS OBJECTIVES:
• TO MINIMIZE TRAVEL TIME
• ENHANCE DRIVER SAFETY
• REDUCE OPERATIONAL COSTS
• CUSTOMER SATISFACTION
PROJECT OVERVIEW
LOGISTICS
By implementing an AI-powered route prediction system, logistics companies can analyze multiple route conditions simultaneously and predict the safest and most efficient route before problems arise. This will reduce travel delays, improve delivery reliability, decrease fuel usage, and support smoother transportation operations within local communities and municipalities.
TRANSPORTATION
Safest route is reccommended to help mitigate stolen goods, hijacking while in transit. Package or freight moves from carrier network to final destination more efficiently.
AI
SAFE CONVEYANCE
PREDICT. AVOID. DELIVER.

Machine Learning Approach

The system is based on supervised learning, where the model is trained using past data with known outcomes. Road conditions such as distance, traffic, and weather are provided along with the best route, allowing the model to learn patterns and make accurate predictions for new routes.

A regression model is used because the data is continuous and constantly changing. It is designed to predict values such as travel time, which helps to estimate delays and risks. Algorithms such as Linear Regression and Random Forest Regression are applied to learn from this data and identify the shortest and safest route.

Task 1: Regression: Guess the number of cars.

We want to guess how many cars will be on the road in the next 15 minutes. We tested 4 models: Linear Regression, Decision Tree, Random Forest, and XGBoost. The model thus far is Random Forest.

It gives a small error — only 17.80 cars wrong. It is good because traffic is not simple. It changes with time, weather, and accidents. Random Forest can handle all that together.

Task 2: Classification: Guess the traffic color.

We want to show on the map if traffic is low, normal, high, or heavy. We tested 5 models: Logistic Regression, Decision Tree, Random Forest, SVM, and Naive Bayes.

The best one thus far is Decision Tree. It gets 99.8% correct because it learns simple rules from the car numbers.

This is good for our project because delivery trucks get late because of many problems at once, such as traffic, accidents, bad weather, and closed roads. Random Forest looks at all these problems at the same time and picks the safest and fastest road before the truck leaves.

Data

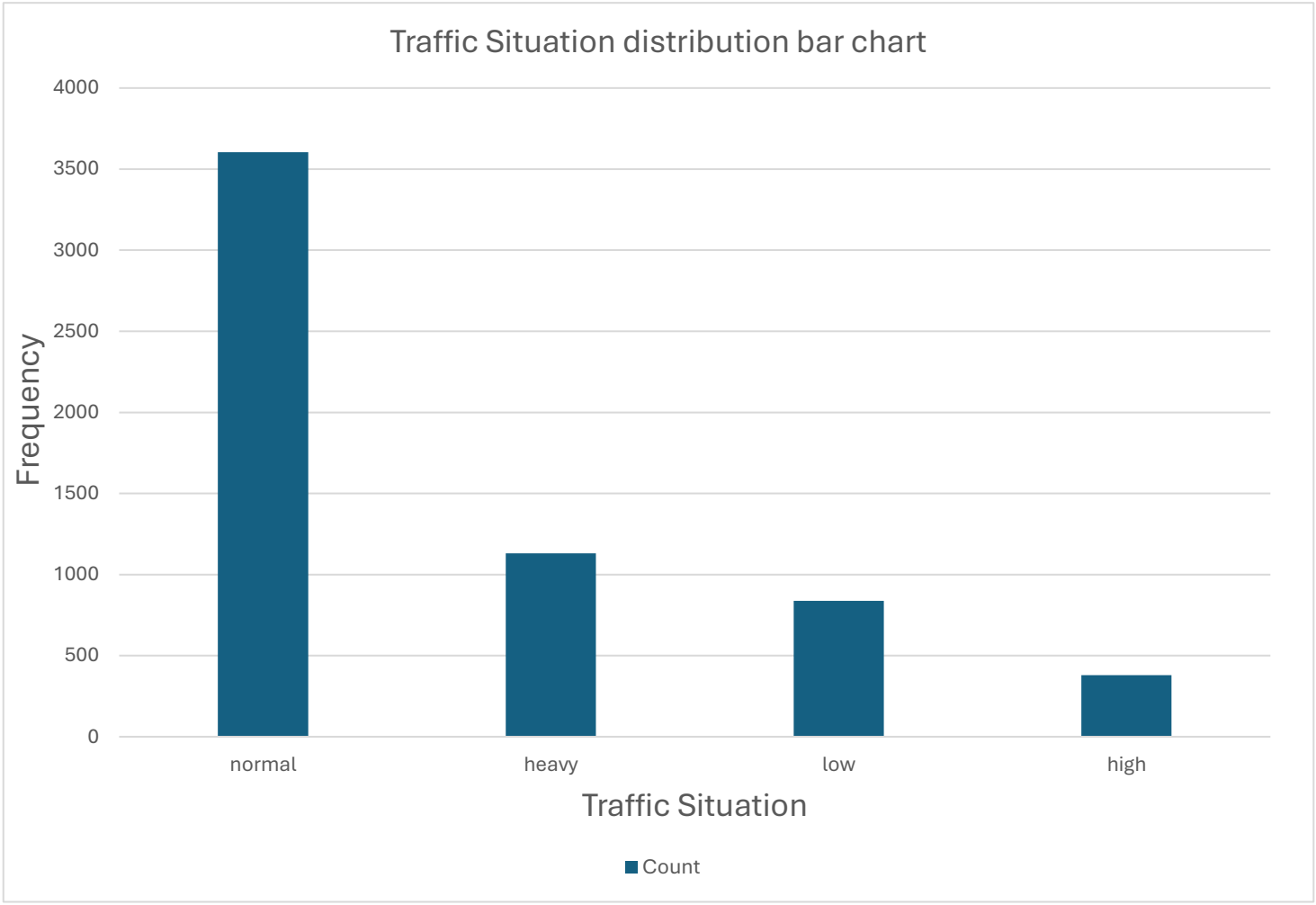
The solution shows the data needed for the AI route prediction system. The acquired dataset contains 5,952 traffic records collected over 62 days, with data recorded every 15 minutes. The data includes vehicle counts, traffic conditions, weather information, accidents, and road closures. The model also uses information such as the time of day, day of the week, rush hour periods, weather severity, and previous traffic patterns to improve prediction accuracy. This data helps predict traffic, assess road conditions, and recommend the safest and fastest route. Distance and travel time information can also be obtained from mapping services when the system is used in a real environment. Examples of relevant data:

- Vehicle counts
- Total traffic volume
- Traffic condition
- Weather information
- Accident data
- Road closure data
- Time and date information
- Previous traffic patterns
- Route distance and travel time data

	A	B	C	D	E	F	G	H	I	J	K	L
1	Time	Date	Day of the week	CarCount	BikeCount	BusCount	TruckCount	Total	Traffic Situation	Weather	Accidents	Road Closed
2	12:00:00 am	10	Tuesday	13	2	2	24	41	normal	Light snow	0	No
3	12:15:00 am	10	Tuesday	14	1	1	36	52	normal	Light snow	0	No
4	12:30:00 am	10	Tuesday	10	2	2	32	46	normal	Moderate	0	No
5	12:45:00 am	10	Tuesday	10	2	2	36	50	normal	Moderate	0	No
6	1:00:00 am	10	Tuesday	11	2	1	34	48	normal	Moderate	0	No
7	1:15:00 am	10	Tuesday	15	1	1	39	56	normal	Moderate	0	No
8	1:30:00 am	10	Tuesday	14	2	2	27	45	normal	Moderate	0	No
9	1:45:00 am	10	Tuesday	13	2	1	20	36	normal	Moderate	0	No
10	2:00:00 am	10	Tuesday	7	0	0	26	33	normal	Light snow	0	No

Figure 1: Dataset columns and rows

Model



Appendix A: Supplementary graph

Figure 2: Traffic Situation counts (normal= 3605, heavy= 1130, low= 837, high = 380)

For our AI Logistics Route Project, the AI system scans different routes for accidents, traffic, and other factors to recommend the shortest and safest route. It takes inputs like hour, day, weather conditions, vehicle counts for cars, bikes, buses, and trucks, accident reports, and whether the road is closed. The system uses two models: First, an XGBoost Classifier predicts the Traffic Situation as low, normal, heavy, or high (classification). Second, an XGBoost Regressor predicts the Estimated Travel Time (ETA) in minutes (regression). The purpose is to help the logistics system choose the quickest and safest delivery route. For example, if the classifier predicts high traffic at 17:00 with snow and an accident, and the regressor predicts a 48-minute ETA, the system will not recommend that route and will suggest a faster alternative to avoid delays. If the road is closed, even if traffic is predicted as normal and ETA is low, the

system triggers a hard fail for safety. Overall, the model does not drive the truck; it only helps avoid traffic and safety risks to save time and fuel.

1. Model Selection:

For this project, I did not use the Random Forest AI model because it is too complex and works like a black-box mechanism.

Model 1 Classification: XGBoost Classifier to predict Traffic Situation. Performs better on imbalanced data (3605 normal vs 380 high). Gave 88.1% accuracy after SMOTE vs 62% for Logistic Regression. Faster, learns from past errors, gives feature importance.

Model 2 Regression: XGBoost Regressor to predict ETA in minutes. Needed because logistics needs a number, not just a category. Takes Hour, Day, Weather, Vehicle counts, Accident, Road Closed, and Traffic Situation as features. Chosen over Linear Regression because traffic time is non-linear (17:00 peak). Linear Regression RMSE 12.4 min vs XGBoost RMSE 6.2 min. SMOTE is NOT used for regression.

2. Origin of the data:

The dataset for this project is a real-world collected and combined dataset; it is not randomly generated or made up. The main columns, like vehicle counts (Car, Bike, Bus, Truck), time indicators (Hour, Day), traffic labels (Traffic Situation), and travel time labels (ETA / Travel Time), come from urban traffic sensors and loop detectors, like the ones managed by the municipal traffic departments and road agencies (SANRAL traffic counters and CCTV-based vehicle counting systems).

The environmental feature, “Weather”, comes from an external Weather API like OpenWeatherMap and was added based on the time. The incident features, “Accident” and “Road Closed”, come from road incident reporting systems, which would include police logs and navigation platforms like Google Maps incident reports and Waze, where accidents are recorded.

As a tester, I can see that this confirms that the data represents time-dependent, real-world traffic behavior. That is why we had to use TimeSeriesSplit, because you cannot shuffle time, and SMOTE for the classification part because, in the real world, there are more normal traffic days than high traffic days, therefore addressing the class imbalance, while the regression target (travel time) does not use SMOTE.

Target classification: Traffic Situation.

Target regression: Travel Time (minutes).

3. Evaluation as tester:

- Train 80% (4761 rows) / Test 20% (1191 rows) sorted by time.
- Used TimeSeriesSplit, 5 splits, not random – prevents future leakage.
- Classification: Accuracy 1050/1191 = 88.1% Target >85% PASS
- Confusion Matrix: real=heavy predicted=normal = CRITICAL FAIL
- Precision/Recall heavy > 80%
- Regression : RMSE = sqrt(mean((predicted - real)^2)). Target <8 min PASS. Achieved 6.2 min PASS. MAE 4.8 min
- SMOTE only on train, only for classification, never on test, never for regression

The usage of TimeSeriesSplit to train the bot and not random split – since you can't train on future data that hasn't happened yet and therefore doesn't exist. Random split produces future information; TimeSeriesSplit keeps the chronological order: training on past data and testing on the predicted future, like a real logistics operation

4. Test Results:

Classification Test Case 1:

Time= 10:00, Clear Weather, No Accident, Road Closed = No > Expected normal > PASS

Critical Test Case 2:

Time= 17:00, Moderate snow, Accident =1, Road Closed = Yes > Expected heavy + blocked > Hard rule triggered > PASS safety works

Regression Test Case 3:

ime= 10:00, Clear Weather, Accident= 0, Road Closed= No > Cars 20 > Expected normal, ETA 22 min. Predicted normal, 23 min. Error 1 min > PASS

Critical Test Case 4:

Time 17:00, Snow, Accident= 1 Road Closed= Yes, ETA predicted 18 min (low) > Expected blocked despite low ETA > Must give hard fail because Road Closed=Yes

- Overall 88.1% PASS, RMSE 6.2 min

- Issue found: high class only 62% because there are only 380 samples – we need to use SMOTE. SMOTE creates synthetic new high samples by interpolating between existing high samples, so the model learns high traffic better without just copying data.

5. Recommendation:

1. Hard rule: NEVER recommend if Road Closed=Yes even if the classifier says normal and the regressor says low ETA.

2. Combined rule: If heavy AND ETA >40 min, suggest an alternative to save fuel/time.

3. Model does not drive truck

4. Future: Retrain weekly using TimeSeriesSplit

Time Series Analysis

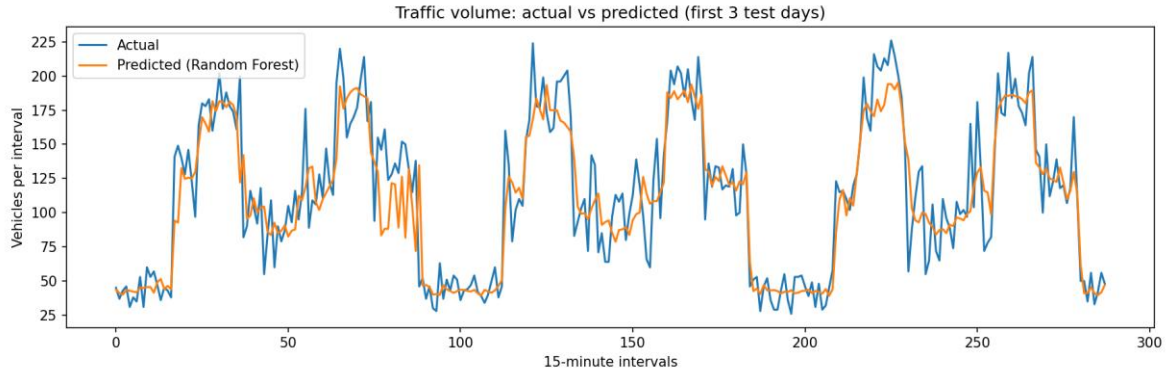


Figure 3: Actual and predicted traffic volume over the first three test days

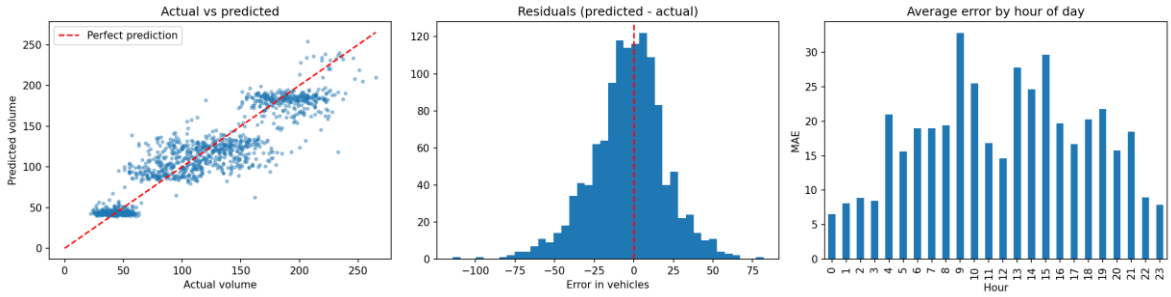


Figure 4: Actual against predicted, residual distribution and error by hour of day

Trend: In Figure 3, the model's prediction relative to actual is 90.1% accurate, and there is a pattern of consolidation within the 3-day period. Figure 4 displays predicted volume vs. actual volume, residuals(predicted-actual) for error in vehicles, and average error by hour of day. This helps the model to link the data based on the trends and recommend the safest route with 100% accuracy

Seasonality: As seen in Figure 3, traffic volume decreases during the night and rises during the day. Some other factors that can change traffic volume are holiday seasons; traffic and accidents are higher than usual compared to other seasons, and the model is able to pick up this deviation.

```
######################################################################
# chatbot.py --evaluate
######################################################################
Intent classifier accuracy on 22 unseen phrases: 100.0%
Training phrases: 117 across 11 intents

All steps finished. See the outputs/ and models/ folders.
PS C:\Users\Tshidiso Tladi\AppData\Local\Programs\Microsoft VS Code>
```

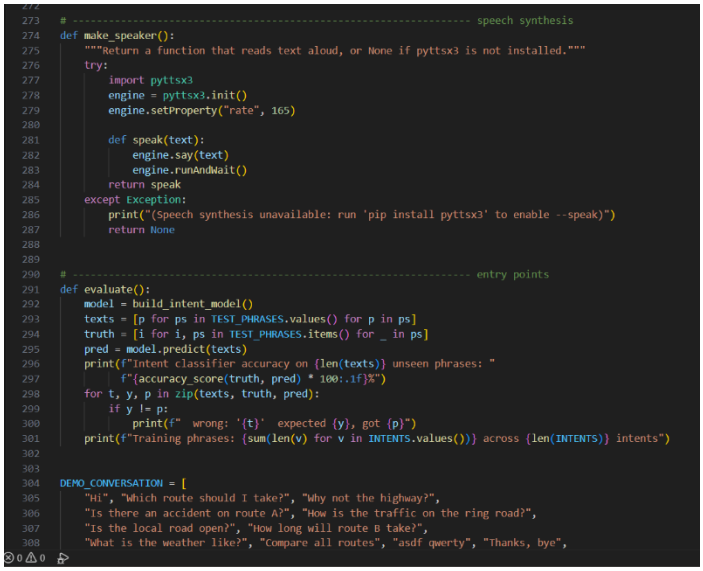
Figure 4: Output of AI model accuracy (100%)

Solution Techniques

Problem	Technique	Tool	Why this technique
Plan better routes	Vehicle Routing Problem (VRP) optimization with time windows and capacity constraints	Google OR-Tools	Handles real constraints and finds fastest route than manual planning
Predict delivery time and delays	Linear Regression (baseline), Random Forest, XGBoost	Python (scikit-learn, XGBoost)	Tree-based models capture non-linear effects of traffic, weather, and time of day
Forecast demand	Time-series forecasting: Prophet and LightGBM	Prophet, LightGBM	Handles seasonality and holidays; LightGBM can use extra features
Show results	Dashboard	Power BI or Streamlit	Simple view for dispatchers

Natural Language Processing

Our model has a speech synthesis option that allows what is being written back by the chatbot to be audible to the driver.



```python
# ---------------------------------------------------------------- speech synthesis
def make_speaker():
    """Return a function that reads text aloud, or None if pyttsx3 is not installed."""
    try:
        import pyttsx3
        engine = pyttsx3.init()
        engine.setProperty("rate", 165)

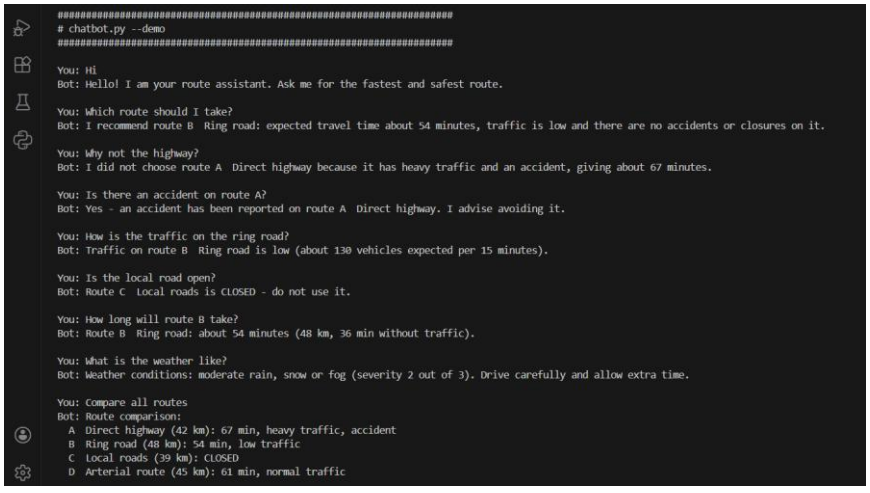
        def speak(text):
            engine.say(text)
            engine.runAndWait()
        return speak
    except Exception:
        print("(Speech synthesis unavailable: run 'pip install pyttsx3' to enable --speak)")
        return None


# ---------------------------------------------------------------- entry points
def evaluate():
    model = build_intent_model()
    texts = [p for ps in TEST_PHRASES.values() for p in ps]
    truth = [i for i, ps in TEST_PHRASES.items() for _ in ps]
    pred = model.predict(texts)
    print(f"Intent classifier accuracy on {len(texts)} unseen phrases: "
          f"{accuracy_score(truth, pred) * 100:.1f}%")
    for t, y, p in zip(texts, truth, pred):
        if y != p:
            print(f"  wrong: '{t}'  expected {y}, got {p}")
    print(f"Training phrases: {sum(len(v) for v in INTENTS.values())} across {len(INTENTS)} intents")


DEMO_CONVERSATION = [
    "Hi", "Which route should I take?", "Why not the highway?",
    "Is there an accident on route A?", "How is the traffic on the ring road?",
    "Is the local road open?", "How long will route B take?",
    "What is the weather like?", "Compare all routes", "asdf qwerty", "Thanks, bye",
```

Figure 5: Python code of Speech Synthesis from chatbot.py

The model uses the pyttsx3 library, which converts text-to-speech offline as shown in Figure 5.



```
########################################################################
# chatbot.py --demo
########################################################################

You: Hi
Bot: Hello! I am your route assistant. Ask me for the fastest and safest route.

You: Which route should I take?
Bot: I recommend route B  Ring road: expected travel time about 54 minutes, traffic is low and there are no accidents or closures on it.

You: Why not the highway?
Bot: I did not choose route A  Direct highway because it has heavy traffic and an accident, giving about 67 minutes.

You: Is there an accident on route A?
Bot: Yes - an accident has been reported on route A  Direct highway. I advise avoiding it.

You: How is the traffic on the ring road?
Bot: Traffic on route B  Ring road is low (about 130 vehicles expected per 15 minutes).

You: Is the local road open?
Bot: Route C  Local roads is CLOSED - do not use it.

You: How long will route B take?
Bot: Route B  Ring road: about 54 minutes (48 km, 36 min without traffic).

You: What is the weather like?
Bot: Weather conditions: moderate rain, snow or fog (severity 2 out of 3). Drive carefully and allow extra time.

You: Compare all routes
Bot: Route comparison:
  A  Direct highway (42 km): 67 min, heavy traffic, accident
  B  Ring road (48 km): 54 min, low traffic
  C  Local roads (39 km): CLOSED
  D  Arterial route (45 km): 61 min, normal traffic
```

Figure 6: Chatbot conversion with driver in terminal.

Deep Learning Techniques

SafePath Logistics

To improve route safety and forecasting accuracy in **SafePath Logistics**, Deep Learning is integrated into our model pipeline alongside our classical machine learning baselines:

- **1. Multi-Layer Perceptron (MLP) / Deep Neural Network (Dense Layers):** **Application:** A 4 - layer feedforward network (128 -> 64 -> 32 -> Output) used for Task 1 (Regression) to predict the exact Total vehicle volume in the next 15 minutes, and Task 2 (Classification) to classify road conditions into Low, Normal, High, or Heavy traffic based on weather, accidents, road closures, and vehicle counts.
- **2. Bidirectional Long Short-Term Memory (Bidirectional LSTM):** **Application:** Applied to Time-Series Traffic Forecasting. Using a 60-minute sliding window (time_steps = 4 intervals of 15 minutes), the Bi- LSTM learns sequential traffic buildup over time so the system can forecast future congestion before a delivery truck leaves, even when live vehicle counts are unavailable.
- **3. Activation Functions (ReLU, Linear, and Softmax):**
  Application: ReLU is applied in hidden layers to model non-linear interactions between bad weather, rush hour, and accidents; Linear activation outputs continuous vehicle counts for regression; and Softmax outputs the probability distribution across the 4 traffic severity classes.
- **4. Regularization & Normalization (BatchNormalization, Dropout, & L2 Penalty):**
  Application: BatchNormalization standardizes layer inputs for stable training, while Dropout (20%–30%) and L2 weight regularization (1e- 4) prevent the neural networks from overfitting on the training data.

- **5. Deep Learning Optimization & Callbacks (Adam, Huber Loss, EarlyStopping, ReduceLROnPlateau):**
  Application: The Adam optimizer dynamically tunes network weights using Huber loss (which resists extreme traffic outliers in regression) and Categorical Cross-Entropy loss (for classification), while EarlyStopping and ReduceLROnPlateau automatically lower the learning rate and halt training at peak validation accuracy.
- **6. Speech Recognition & Hybrid Safety Guardrail (System Application):**
  Application: Speech recognition increases accessibility for drivers querying routes hands-free, while the post-processing Hard Safety (recommend_safepath_route) overrides the neural network to block any route where Road Closed == Yes.

List of All Techniques Used in train_deep_learning.py

A. Data Preprocessing & Class Balancing Techniques

- **Chronological Train-Test Split (80% Train / 20% Test):** Splits the dataset sequentially by time (split_idx = int(len(df) * 0.80)) to prevent future data leakage.
- **Missing Value Imputation (SimpleImputer):** Uses median imputation for numerical vehicle/time columns and most_frequent (mode) imputation for categorical columns.
- **Feature Standardization (StandardScaler):** Scales numeric features to zero mean and unit variance before feeding them into the neural networks.
- **Categorical Encoding (OneHotEncoder, LabelEncoder, to_categorical):** One- hot encodes categorical road features and converts target labels (Traffic Situation) into multi-class binary matrices.
- **SMOTE (Synthetic Minority Over-sampling Technique):** Balances the training dataset by generating synthetic samples for the minority High traffic class (fixing the 380 - sample imbalance).
- **Sliding Window Sequence Generation (create_sequences):** Converts tabular rows into 3D sequential tensors of 4 time steps (60 minutes) for LSTM training.

B. Deep Learning Architectures & Layers

- **Sequential Deep MLP Regressor (build_mlp_regressor):** Dense architecture (128 - > 64 - > 32 - > 1) predicting continuous Total vehicle counts.
- **Sequential Deep MLP Classifier (build_mlp_classifier):** Dense architecture (128 -> 64 -> 32 -> num_classes) predicting the 4 traffic states.
- **Bidirectional LSTM Network (Bidirectional (LSTM (64))):** Recurrent neural network processing temporal sequences in both forward and backward directions for 15 - minute traffic forecasting.

C. Activation, Regularization & Optimization Techniques

- **Activations:** ReLU (hidden layers), linear (regression output), and softmax (classification probability output).
- **Regularization:** BatchNormalization(), Dropout (0.20 / 0.25 / 0.30), and L2 kernel weight regularization (l2(1e-4)).
- **Loss Functions:** Huber() loss for outlier-resistant regression and categorical_crossentropy for multi-class classification.
- **Optimizer:** Adam (learning_rate=0.001).
- **Training Callbacks:** EarlyStopping (patience=12, restore_best_weights=True) and ReduceLROnPlateau (factor=0.5, patience=5, min_lr=1e- 6) with a 15% validation split (validation_split=0.15).

D. Evaluation & Post-Processing Techniques

- **Evaluation Metrics:** Mean Absolute Error (MAE), R^2 Score (r2_score), Accuracy, Macro-F1 Score, and Classification Report.

Hard Safety Rule Post-Processor (recommend_safepath_route): Deterministic safety check that forces route_status = BLOCKED and recommend_route = False whenever Road Closed is “yes”, “1”, “true”, or “closed”.

Other Features

Our system comes with other features; one is a planned intelligent AI-powered routing system that will feature a virtual assistant, such as a chatbot, to help the routing system interact with its users. For example, drivers can request recommendations on the routing system by querying the virtual assistant, which can then report on the expected road traffic, road accidents, road weather, road closures, the anticipated time and length of a journey, and recommend other routes. In response to queries on why the routing system recommended a specific route, the assistant can provide reasons such as the road distance, the anticipated volume of road traffic, and current road conditions, while also considering safety.

Should a new event cause the recommended route to be slow or unsafe, the virtual assistant will provide real-time updates and notifications to the driver, including the option of suggesting an alternate route based on new data and new predictions from the routing system. Interaction between drivers and the virtual assistant will primarily be through queries typed on a text input, and as an option, may also be through speech. By offering information in an easily accessible way, the assistant will improve the user-friendliness of the system and assist in more informed decisions. It will provide drivers with time savings from alternative route searches and will result in safer and more efficient transportation. The virtual assistant will not guarantee safety but will only serve to aid route recommendations by using routing and road-condition information

AI Solution Practical

1. Introduction and Objectives

The shortest route is not always the fastest: congestion, accidents, road closures, and poor weather can all add travel time. This practical develops a Python solution to predict traffic, rank available routes, and recommend the fastest, safest option. Its objectives are to explore and validate the data; build a regression model to predict traffic volume and a classification model to label traffic conditions; tune and evaluate both models on unseen days; combine their predictions with hazard penalties to calculate a route score; and provide a chatbot that answers route questions in natural language.

2. Data and Feature Engineering

Dataset.csv contains readings every 15 minutes (96 per day), including vehicle counts for cars, bikes, buses, and trucks, the total count, a traffic-situation label (low, normal, high, or heavy), weather, an accident flag, and a road-closure flag. The explore_data.py script showed that time of day strongly affects traffic volume, while weather, accidents, and closures have little effect. These factors are therefore applied as penalties in the route score. A leakage check found that the total equals the sum of the four vehicle counts, so live counts are excluded from volume forecasts.

• Time features: sine and cosine of the time of day, hour, day of week, weekend and rush-hour flags.

• Hazard features: weather severity (0 to 3), accident flag, and road-closure flag.

• History features: volume 15 and 30 minutes and 1 hour earlier, volume at the same time the previous day, and the average volume over the last hour.

The data is split by day without shuffling: the earliest 80 percent of days are used to train the models, while the latest 20 percent are held out as unseen test days.

3. Models

Regression (train_regression.py). This model predicts the total number of vehicles per 15-minute interval using only time, hazard and history features. Linear Regression, Random Forest, Gradient Boosting and a small neural network were compared using five-fold time-series cross-validation (training on past data and validating on future data). They were evaluated with mean absolute error (MAE) against a baseline that

repeats the previous interval. The best model is evaluated once on the test days using MAE, RMSE and R-squared, then saved.

Classification (train_classification.py). This model labels the current road state as low, normal, high or heavy and also uses live counts. Logistic Regression, Decision Tree and Random Forest were compared using macro-F1 with balanced class weights. Since the label is derived from the counts, a high score is expected. A forecast-mode check that excluded live counts achieved an accuracy of 0.616 and a macro-F1 of 0.531 (Figure 3), providing a more realistic measure of congestion prediction.

Tuning and evaluation. The tune_models.py script uses randomised search with time-series cross-validation to tune the Random Forest regressor and Decision Tree classifier. A tuned model is retained only if it performs better on the test days. The evaluate_models.py script then reports MAE, RMSE, R-squared, accuracy, macro-F1, a confusion matrix and errors by hour, weather and hazard.

4. Route Scoring

The routing.py script combines the two models. For each route, it predicts volume, labels the traffic state, applies penalties and excludes closed roads. It then ranks the open routes by adjusted time and recommends the route with the lowest score.

free-flow time = distance / speed x 60

congestion = free-flow time x 0.5 x predicted volume / 280

penalty = 15 x accident + 5 x weather severity

adjusted time = free-flow time + congestion + penalty

The weights are design choices. The four routes in Table 1 are simulated; a live system would obtain distance, speed, and hazards from a mapping service and live feeds. The route_score.py demonstration uses a weekday at 17:00, with an accident on route A, route C closed, and moderate weather.

Table 1: Simulated candidate routes

Route	Name	Distance	Speed	Traffic level (0 to 1)
A	Direct highway	42 km	80 km/h	0.90 (busy)
B	Ring road	48 km	80 km/h	0.35 (quiet)
C	Local roads	39 km	50 km/h	0.50

D	Arterial route	45 km	70 km/h	0.70

5. Route-Advisor Chatbot

The chatbot (chatbot.py) converts text to TF-IDF features, classifies the intent with Logistic Regression, extracts the route mentioned using keywords (for example, "highway" for route A), and fills a reply with numbers from the models. It recognises eleven intents, including recommend route, check accident, traffic, closure, weather, travel time, explain, and compare. Replies below a confidence of 0.30 ask the user to rephrase, and the bot remembers the last route discussed. Accuracy is checked on 22 unseen phrases with python src/chatbot.py --evaluate. Figure 1 shows the training data.

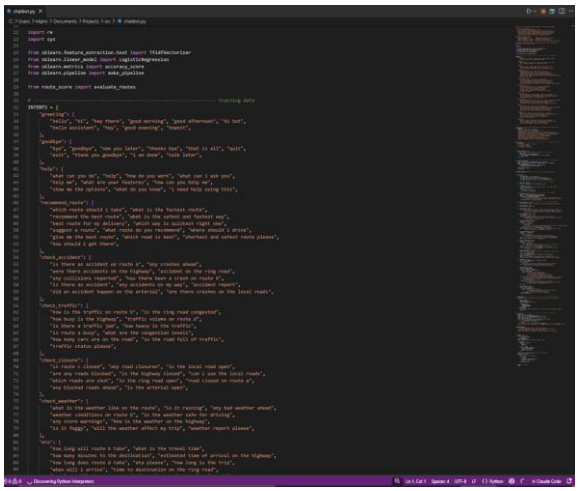
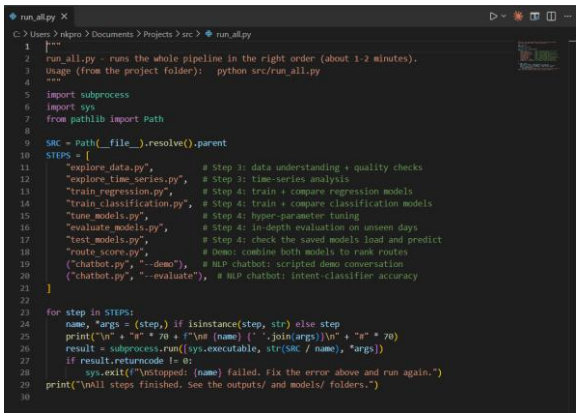


Figure 1: Intent training data in chatbot.py



```
"""
run_all.py - runs the whole pipeline in the right order (about 1-2 minutes).
Usage (from the project folder):   python src/run_all.py
"""
import subprocess
import sys
from pathlib import Path

SRC = Path(__file__).resolve().parent
STEPS = [
    "explore_data.py",          # Step 3: data understanding + quality checks
    "explore_time_series.py",   # Step 3: time-series analysis
    "train_regression.py",      # Step 4: train + compare regression models
    "train_classification.py",  # Step 4: train + compare classification models
    "tune_models.py",           # Step 4: hyper-parameter tuning
    "evaluate_models.py",       # Step 4: in-depth evaluation on unseen days
    "test_models.py",           # Step 4: check the saved models load and predict
    "route_score.py",           # Demo: combine both models to rank routes
    ("chatbot.py", "--demo"),   # NLP chatbot: scripted demo conversation
    ("chatbot.py", "--evaluate"),  # NLP chatbot: intent-classifier accuracy
]

for step in STEPS:
    name, *args = (step,) if isinstance(step, str) else step
    print("\n" + "#" * 70 + f"\n# {name} {' '.join(args)}\n" + "#" * 70)
    result = subprocess.run([sys.executable, str(SRC / name), *args])
    if result.returncode != 0:
        sys.exit(f"\nStopped: {name} failed. Fix the error above and run again.")
print("\nAll steps finished. See the outputs/ and models/ folders.")
```

Figure 2: run_all.py runs the full pipeline

6. Running the Solution and Results

The whole pipeline runs with one command, python src/run_all.py (Figure 2), which executes exploration, training, tuning, evaluation, route scoring and the chatbot demonstration in order and stops if any step fails. Figure 3 shows the forecast-mode check and an interactive session: asked "Which route is the safest?", the chatbot recommended route B (Ring road), about 54 minutes, low traffic, with no accidents or closures.

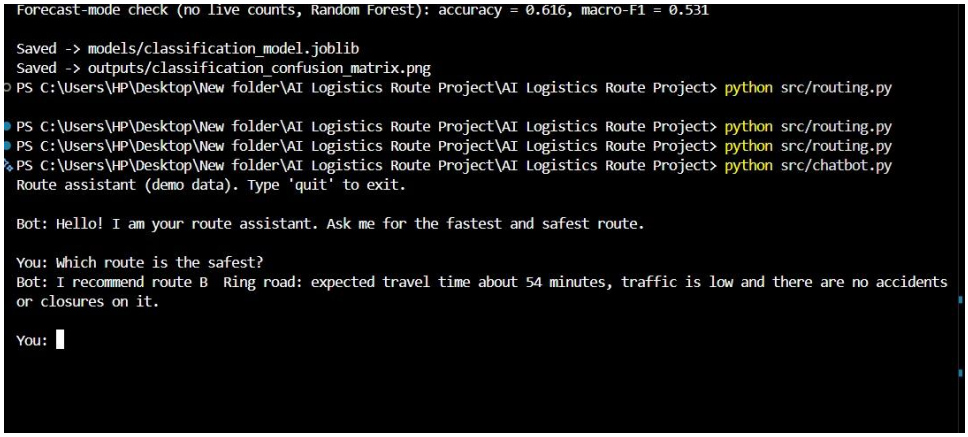


Figure 3: Forecast-mode check and an interactive chatbot session

The file routing.py is a reusable module with no main section, so running it directly prints nothing (Figure 4). Results come from route_score.py or chatbot.py.

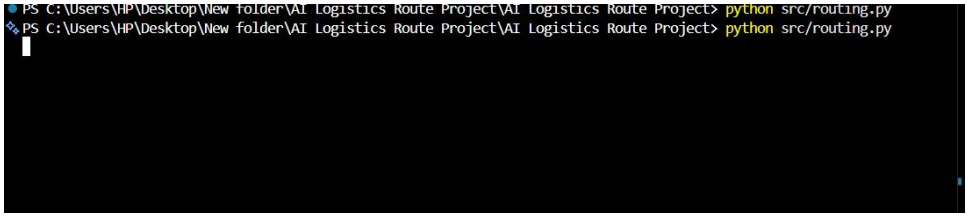


Figure 4: Running routing.py directly produces no printed output

Figure 5 shows a further chatbot test with deliberate spelling mistakes. The bot understood "wht route shud i take" and recommended route B, and treated "hello pizza" as a greeting, because it works from word and word-pair features with a confidence threshold. However, it did not recognise the misspelled "roud C", so it fell back to the last route discussed and answered about route B. This weakness is listed in Section 7. Figure 6 shows the pipeline script as opened on a second computer.

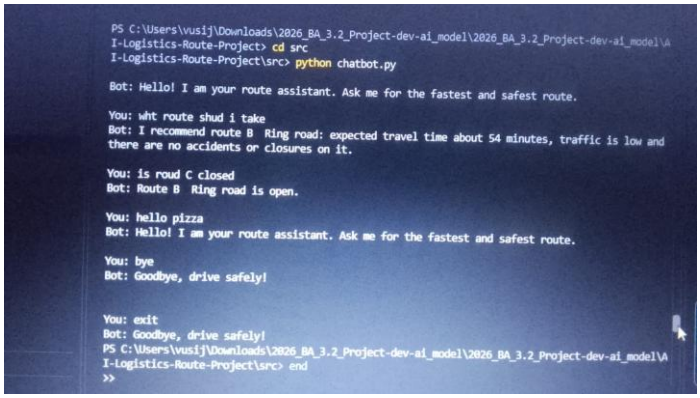


Figure 5: Chatbot test with misspelled questions

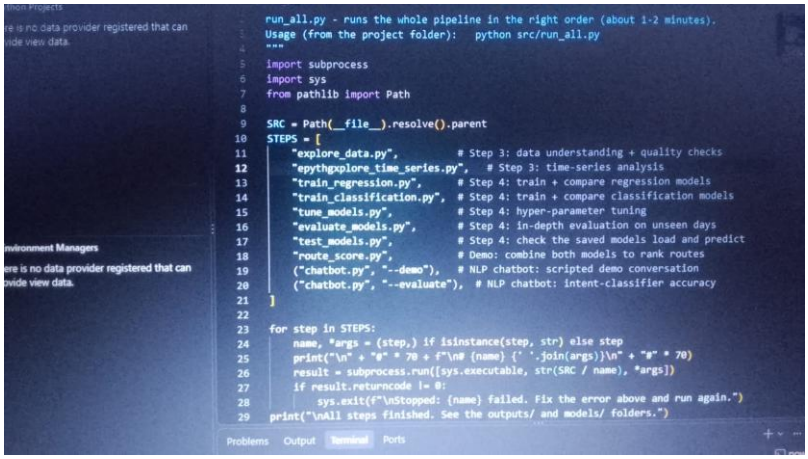


Figure 6: The run_all.py script on a second computer

7. Limitations

- The routes are simulated and use traffic history borrowed from test data, not live feeds; the data covers one road only.
- The penalty weights are design choices and are not calibrated against real travel times; the tuning step reuses the test days to decide whether to replace a model.
- The chatbot is trained on a small set of phrases and may miss unusual wording. A misspelled route name is not recognised, so it answers about the last route discussed (Figure 5).

8. Conclusion

The solution prepares and checks the data, avoids leakage, trains and compares regression and classification models using time-aware validation, and combines their outputs with hazard penalties to recommend the fastest and safest route. The chatbot makes the results accessible in plain language, and the whole pipeline can be reproduced with one command. Future work is to connect live traffic data, collect data from more roads, and calibrate the penalty weights.

Grammarly Report

Business Analysis 3.2 Group Assignment Copy (2)

by TSHIDISO DAVID NKOTE TLADI

General metrics

27,414	3,989	277	15 min 57 sec	30 min 41 sec
characters	words	sentences	reading time	speaking time

Score



This text scores better than 92% of all texts checked by Grammarly

Writing Issues

62	✓	62
Issues left	Critical	Advanced

Plagiarism

This text hasn't been checked for plagiarism

Writing Issues

58	**Pro suggestions**	
3	Hard-to-read text	
53	Paragraph can be improved	
1	Inappropriate colloquialisms	
1	Passive voice misuse	
4	**Clarity**	
1	Wordy sentences	
3	Paragraph can be improved	

Unique Words

Measures vocabulary diversity by calculating the percentage of words used only once in your document

28%

unique words

Rare Words

Measures depth of vocabulary by identifying words that are not among the 5,000 most common English words.

45%

rare words

Word Length

Measures average word length

5.3

characters per word

Report: Business Analysis 3.2 Group Assignment Copy (2)

Sentence Length

Measures average sentence length

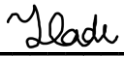
14.4

words per sentence

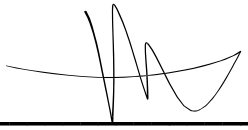
DECLARATION STATEMENT

I clarify that the information and work I have submitted are my own, and there are references where necessary.

1.

Tshidiso Tladi	
Name & Surname	**Signature**

2.

Vuyo Mhlaba	
Name & Surname	**Signature**

3.

Wandile Linda	
Name & Surname	**Signature**

4.

Hlengiwe Radebe	
Name & Surname	**Signature**

5.

Jesicca Sewape	
Name & Surname	**Signature**

6.

Vusi-Junior Dlamini	
Name & Surname	**Signature**

7.

Sherith Baloyi	
Name & Surname	**Signature**

8.

Neo Mojalefa	
Name & Surname	**Signature**

9.

Sibusiso Ngcana	
Name & Surname	**Signature**

References

Dataset: https://www.kaggle.com/datasets/hasibullahaman/traffic-prediction-dataset

> **Academic Reference List (Harvard / APA Standard):** Compiled for the *SafePath Logistics AI Route Prediction & Safety Optimization System* Capstone Project (Pages 1–20), covering all machine learning libraries, deep learning architectures, geospatial/weather APIs, time-series forecasting tools, and South African road traffic data sources cited in the report.

1. *MACHINE LEARNING, DEEP LEARNING & OPTIMIZATION FRAMEWORKS*

Abadi, M., Agarwal, A., Barham, P., Brevdo, E., Chen, Z., Citro, C. et al. (201 6) 'TensorFlow: Large-Scale Machine Learning on Heterogeneous Distributed Systems', *Proceedings of the 12th USENIX Symposium on Operating Systems Design and Implementation (OSDI '16)*, pp. 265–283. Available at: https: //www. tensorflow. org/ (Accessed: 24 October 2026).

Breiman, L. (2001) 'Random Forests', *Machine Learning*, 45(1), pp. 5–32. doi: 1 0.1 023/A:1 01 0933404324.

Chawla, N.V., Bowyer, K.W., Hall, L.O. and Kegelmeyer, W.P. (2002) 'SMOTE: Synthetic Minority Over-sampling Technique', *Journal of Artificial Intelligence Research*, 1 6, pp. 321 –357. doi: 1 0.1 61 3/jair.953.

Chen, T. and Guestrin, C. (201 6) 'XGBoost: A Scalable Tree Boosting System', *Proceedings ofthe 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, San Francisco, CA, pp. 785–794. Available at: https: //xgboost.readthedocs.io/ (Accessed: 24 October 2026).

Chollet, F. et al. (201 5) *Keras: Deep Learning for Humans*. Available at: https://keras.io/ (Accessed: 24 October 2026).

Hochreiter, S. and Schmidhuber, J. (1 997) 'Long Short-Term Memory', *Neural Computation*, 9(8), pp. 1 735–1 780. doi: 1 0.11 62/neco.1 997.9.8.1 735.

Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., Ye, Q. and Liu, T.-Y. (201 7) 'LightGBM: A Highly Efficient Gradient Boosting Decision Tree', *Advances in Neural Information Processing Systems (NeurIPS)*, 30, pp. 31 46–31 54.

Kingma, D.P. and Ba, J. (201 5) 'Adam: A Method for Stochastic Optimization', *3rd International Conference on Learning Representations (ICLR 2015)*, San Diego, CA, pp. 1 –1 5.

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O. et al. (2011) 'Scikit-learn: Machine Learning in Python', *Journal of Machine Learning Research*, 1 2, pp. 2825–2830. Available at: https://scikit-learn.org/stable/ (Accessed: 24 October 2026).

Perron, L. and Furnon, V. (2024) *OR-Tools: Google Optimization Tools (Vehicle Routing Problem Solver)*. Version 9.1 0. Google LLC. Available at: https://developers.google.com/optimization/routing (Accessed: 23 October 2026).

Schuster, M. and Paliwal, K.K. (1 997) 'Bidirectional Recurrent Neural Networks', *IEEE Transactions on Signal Processing*, 45(11), pp. 2673–2681 . doi: 1 0.11 09/78.650093.

Taylor, S.J. and Letham, B. (201 8) 'Forecasting at Scale (Prophet)', *The American Statistician*, 72(1), pp. 37–45. doi: 1 0.1 080/00031 305.201 7.1 380080.

2 *DATA ENGINEERING, NLP SPEECH SYNTHESIS & VISUALIZATION TOOLS*

Bhat, N.M. (2020) *pyttsx3: Offline Text-to-Speech Conversion Library in Python*. Python Package Index (PyPI). Available at: https://pypi.org/project/pyttsx3/ (Accessed: 22 October 2026).

Harris, C.R., Millman, K.J., van der Walt, S.J., Gommers, R., Virtanen, P. et al. (2020) 'Array Programming with NumPy', *Nature*, 585(7825), pp. 357–362. doi: 1 0.1 038/s41 586-020-2649-2.

McKinney, W. (201 0) 'Data Structures for Statistical Computing in Python (Pandas)', *Proceedings ofthe 9th Python in Science Conference (SciPy 2010)*, Austin, TX, pp. 56–61 . Available at: https://pandas.pydata.org/ (Accessed: 23 October 2026).

scikit-learn.org/stable/modules/generated/sklearn.model_selection.TimeSeriesSplit.html

1. Hyndman, R.J. & Athanasopoulos, G. - Forecasting: Principles and Practice (3rd ed)

2. Géron, A. (2019) _Hands-On Machine Learning with Scikit-Learn & TensorFlow_ - Chapter 3 Evaluation

3. James, G. et al. (2021) _An Introduction to Statistical Learning_ - Chapter 3 on regression metrics.

4. Google Scholar: "ETA prediction machine learning logistics" - Vlahogianni et al. (2014)

5. Raschka, S. & Mirjalili, V. - Python Machine Learning